

# Phase 0: Project Foundation

Job Agent | Documentation

---

## 1. Purpose

Phase 0 sets up everything the rest of the project stands on: the folder structure, version control, an isolated Python environment, configuration handling, logging, and a minimal FastAPI server with a health-check endpoint. No business logic is built yet. The goal is a running, committed skeleton that every later phase can build on.

## 2. What was built

- Project folder structure with Python packages for the backend
- Git repository with a **.gitignore** that keeps secrets out of version control
- Virtual environment and a pinned **requirements.txt**
- Settings loaded from a **.env** file via a central config module
- Logging to the console and to a rotating log file
- FastAPI app with CORS configured and a **/health** endpoint
- A one-click start script, **run\_api.cmd**
- Project pushed to a private GitHub repository with a README

## 3. Tech used in this phase

| Package           | Role                                                                 |
|-------------------|----------------------------------------------------------------------|
| fastapi           | Web framework for the API                                            |
| uvicorn[standard] | Server that runs the FastAPI app (--reload restarts on code changes) |
| python-dotenv     | Reads variables from .env files                                      |
| pydantic-settings | Turns .env values into a typed Settings object                       |
| loguru            | Simple, structured logging                                           |

## 4. Project structure

```
Job_Agent/
+-- .gitignore
+-- README.md
+-- run_api.cmd          start script (project root)
+-- backend/
| +-- venv/              virtual environment (not committed)
| +-- app/
| | +-- __init__.py
| | +-- main.py          FastAPI entry point
| | +-- config.py        loads settings from .env
| | +-- api/             route modules (later phases)
| | +-- agent/           Gmail scanner and AI logic (Phase 3)
| | +-- models/          data models (Phase 1)
| | +-- schemas/         request/response schemas (Phase 1)
| | +-- utils/
| | +-- logger.py        logging setup
+-- tests/
+-- .env                 real settings and secrets (not committed)
+-- .env.example         template (committed)
+-- requirements.txt
+-- frontend/           React app (Phase 5)
+-- docs/
```

The virtual environment sits *beside* the code in **backend/**, never around it. It is disposable and can be rebuilt at any time from requirements.txt. Every folder under **app/** contains an empty `__init__.py` so Python treats it as a package.

## 5. Key files

### **.gitignore (project root)**

Prevents secrets and generated files from being committed. The most important entries are **.env**, **credentials.json** and **token.json**.

```
.env
credentials.json
token.json
venv/
__pycache__/
*.pyc
logs/
node_modules/
dist/
```

### **backend/.env and .env.example**

**.env** holds the real values and is never committed. **.env.example** lists the same variable names with safe values and *is* committed, so the setup can be recreated on any machine. Later phases add the OpenAI, Supabase and Telegram keys here.

```
APP_NAME=Job Agent
DEBUG=True
```

### **backend/app/config.py**

One shared **settings** object used by the whole app. Field names match .env variables case-insensitively (APP\_NAME becomes settings.app\_name). Defaults apply when a value is missing.

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "Job Agent"
    debug: bool = True

    # Read the .env file; ignore keys we have not defined yet
```

```
model_config = SettingsConfigDict(env_file=".env", extra="ignore")
```

```
settings = Settings()
```

### ***backend/app/utils/logger.py***

Configures Loguru: verbose console output in debug mode, plus a log file that rotates at 5 MB and keeps seven days of history.

```
import sys
from loguru import logger

def setup_logger(debug: bool):
    logger.remove() # drop the default handler to control format and level
    logger.add(sys.stderr, level="DEBUG" if debug else "INFO")
    logger.add("logs/app.log", rotation="5 MB", retention="7 days", level="INFO")

__all__ = ["logger", "setup_logger"]
```

### ***backend/app/main.py***

The application entry point. Logging is configured first. CORS allows the React dev server (Vite, port 5173) to call the API, and both **localhost** and **127.0.0.1** are listed because browsers treat them as different origins.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.config import settings
from app.utils.logger import logger, setup_logger

setup_logger(settings.debug)

app = FastAPI(title=settings.app_name)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/health")
def health():
    logger.info("Health check called")
    return {"status": "ok", "app": settings.app_name}
```

### ***run\_api.cmd (project root)***

Bundles the three start-up steps into one file. **call** is required before the activate script, otherwise CMD never returns to run the next line. **app.main:app** means the variable *app* inside *app/main.py*.

```
@echo off
cd backend
call venv\Scripts\activate
uvicorn app.main:app --reload --port 8000
```

## **6. Setup commands (summary)**

```
:: folders and packages
cd C:\Users\User\Desktop\COD
mkdir Job_Agent & cd Job_Agent
mkdir backend frontend docs
cd backend
```

```

mkdir app app\api app\agent app\models app\schemas app\utils tests

:: virtual environment and dependencies
python -m venv venv
venv\Scripts\activate
pip install fastapi "uvicorn[standard]" python-dotenv loguru pydantic-settings
pip freeze > requirements.txt

:: environment file
copy .env.example .env

:: git
cd ..
git init
git add .
git commit -m "Phase 0: project foundation with health check"

```

The empty `__init__.py` files are created with **type nul > app\\_\_init\_\_.py** (repeated for each package folder).

## 7. Running and verifying

Start the server with **run\_api.cmd**, leave the window open, then check:

| Check                        | Expected result                                     |
|------------------------------|-----------------------------------------------------|
| http://127.0.0.1:8000/health | {"status":"ok","app":"Job Agent"}                   |
| http://127.0.0.1:8000/docs   | Swagger UI listing the /health endpoint             |
| CMD window                   | A "Health check called" log line after each request |
| backend\logs\app.log         | File exists and contains the same log line          |

## 8. GitHub

- Private repository created with no README, .gitignore or license (they already exist locally)
- Connected with **git remote add origin** and pushed with **git push -u origin main**
- Git does not track empty folders. frontend/, docs/ and tests/ only appear on GitHub once they contain a file (or a .gitkeep placeholder)
- Before every commit, run **git status** and confirm .env, venv and logs are not listed
- Routine after each phase: **git add .**, **git commit -m "..."**, **git push**

## 9. Troubleshooting

| Problem                                    | Cause and fix                                                               |
|--------------------------------------------|-----------------------------------------------------------------------------|
| 'uvicorn' is not recognized                | Venv not active or packages missing. Activate it and reinstall the packages |
| No module named 'app'                      | Uvicorn started from the wrong folder. It must run from inside backend      |
| No module named pydantic_settings / loguru | Package missing. pip install it with the venv active                        |
| Address already in use                     | An old server is on port 8000. Close it or use --port 8001                  |
| .env or .gitignore saved as .txt           | Windows added an extension. Check with dir /a and rename the file           |
| .env changes not picked up                 | Uvicorn does not reload for .env. Stop (Ctrl+C) and restart the server      |

## 10. Outcome and next phase

At the end of Phase 0 the server starts with one click, **/health** responds, logs are written to the console and to file, secrets are excluded from Git, and the code is on GitHub. **Phase 1** adds the data models, Pydantic schemas and JWT authentication (signup, login, logout, and a protected-route dependency).